

PRACTICAL SESSION 5:

LEARNING IN BIOINSPIRED DECISION MAKING



AUTHOR: IGNACIO GONZÁLEZ MELÉNDEZ (100522976)

COORDINATOR: MARCOS MAROTO GÓMEZ

COURSE: INTELLIGENT DECISION-MAKING IN ROBOTICS

EXERCISE 1:

After completing the required part of the code and then executing it, the optimal policy I got can be seen in Figure 1. This policy represents the fact that the robot is trying to avoid the obstacle in (2,2) due to the fact that it gives a penalty of -10. Thus, our robot will get to the goal position avoiding the obstacle.

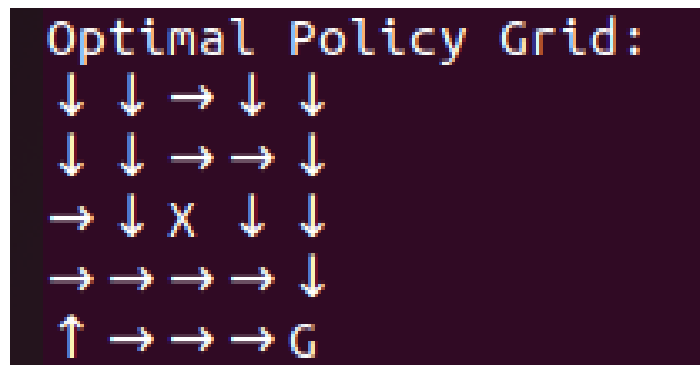


Figure 1

EXERCISE 2:

In this exercise, we increase the number of obstacles as the practice said. In Figure 2 we can see the optimal policy for this case. However, we can see that there are some situations in which our robot does not know what to do, for example in position (2,0) it is trying to go left where there is a wall. To solve this, I decided to implement a dynamic ϵ -greedy starting with a high exploration and decreasing it to exploit the best actions in the end. After doing this, I reached a new optimal policy in which the robot is able to reach the goal avoiding obstacles from every position, look at Figure 3.

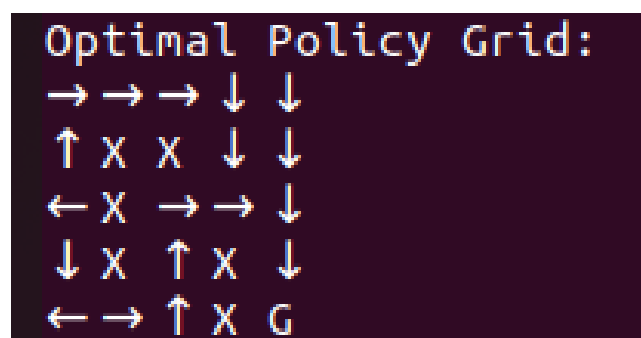


Figure 2

Now, in Figure 3 we have an optimal policy with the previous problem solved.

```

Trained Q-table:
q-values for state (i, j): Q((i,j),↑), Q((i,j),↓), Q((i,j),←), Q((i,j),→)
q-values for state (0, 0): [-3.1301047130649526, -3.1501341142063346, -3.1497983213194183, -1.6127966468154833]
q-values for state (0, 1): [-2.5183305097477504, -9.709847411056998, -2.5003177528468923, -0.1865644319361739]
q-values for state (0, 2): [-1.781501007293764, -8.050733891325054, -1.97522421689246, 1.3249993762958359]
q-values for state (0, 3): [-1.2406538054168998, 2.8842082652234353, -1.2311504143824328, -1.1586734170064887]
q-values for state (0, 4): [-0.8201010811141989, -0.4860442330087874, -0.9298115835589242, -0.7658303981861421]
q-values for state (1, 0): [-2.9813757437992914, -2.908897473219228, -2.9600846541905463, -8.960386649773115]
q-values for state (1, 1): [-1.6411915019920538, -4.855466647531, -1.6961765373204392, -7.35547529381572]
q-values for state (1, 2): [-1.103921817590378, -0.969899207066651, -3.5627767888819135, -0.7884097465775128]
q-values for state (1, 3): [-0.7821967171806237, 4.48883385018844, -7.500538458971785, -0.36236351059808725]
q-values for state (1, 4): [-0.6321704250082969, 2.849028248746269, -0.49392045991617395, -0.29266104377053415]
q-values for state (2, 0): [-2.6860713192573513, -2.6136431335993904, -2.622867105816917, -7.832164461336039]
q-values for state (2, 1): [-6.864425491640191, -3.5519929021900007, -0.9473791734862538, -0.8243215615716779]
q-values for state (2, 2): [-2.799418037644755, -1.1324236613379837, -5.913600484820404, 1.1333148900735543]
q-values for state (2, 3): [-0.37798199110511893, -4.583350658800001, -0.8052223214298059, 6.173851703702918]
q-values for state (2, 4): [-0.35331752207911976, 7.995369792711639, 0.417522287821514, 1.0939272343045612]
q-values for state (3, 0): [-2.4249779410049523, -2.3528634878750476, -2.3598618962020756, -7.508287723544631]
q-values for state (3, 1): [-5.356308586951, -0.9982385529281124, -0.8487370830875505, -0.9035152406661143]
q-values for state (3, 2): [-0.985142115434783, -1.155206165637857, -6.361914690949208, -3.3901350499]
q-values for state (3, 3): [-0.26144714327330365, -3.2680000000000002, -0.38721782979100006, 2.946798836118946]
q-values for state (3, 4): [1.3065322238072736, 9.999500200419472, -4.237137985200792, 1.8110429932881154]
q-values for state (4, 0): [-2.1410699186574234, -2.1253803716532573, -2.102598593057025, -2.1526481983253385]
q-values for state (4, 1): [-7.0367544878410015, -1.8932924236476532, -1.9093723797193127, -1.8513656047062466]
q-values for state (4, 2): [-1.4975487601347757, -1.485422289051244, -1.4950160329243327, -5.165965999]
q-values for state (4, 3): [-4.38375051861086, -3.8512, -0.21592900000000004, 4.68559]
q-values for state (4, 4): [0.0, 0.0, 0.0, 0.0]
Optimal Policy Grid:
→→→↓↓
↓X X ↓↓
↓X →→↓
↓X ↑X ↓
←→↓X G

```

Figure 3

EXERCISE 3:

In this case, our robot is not able to reach an obstacle state but if he attempts to do it a reward of -10 will be given. Thus, in Figure 4 we can see the Q-table generated following these requirements.

If we compare it with the Q-table from the previous exercise (Figure 3) we can clearly see that the main difference consist on that now those obstacle states do not have any value in the Q-table, this is because the robot cannot get to them, so there is no step in which we have information about reaching them.

```

Trained Q-table:
q-values for state (i, j): Q((i,j),↑), Q((i,j),↓), Q((i,j),←), Q((i,j),→)
q-values for state (0, 0): [-3.664376502043355, -3.630451649833196, -3.6600835231406097, -1.4157785762328368]
q-values for state (0, 1): [-3.0905089747699646, -11.249817723988494, -3.1150042205142614, -0.002016674634742345]
q-values for state (0, 2): [-2.4893562160176055, -10.889972304589966, -2.5757079598856096, 1.467328848008296]
q-values for state (0, 3): [-1.9873791799362404, -1.7416285316066031, -1.8433050317360147, 2.970006004932423]
q-values for state (0, 4): [-1.4808750047745283, 4.5270835443703, -1.3789834692088758, -1.5915223964990368]
q-values for state (1, 0): [-3.476473385935796, -3.4559085952292508, -3.452640473573957, -11.317496484676735]
q-values for state (1, 1): [0.0, 0.0, 0.0, 0.0]
q-values for state (1, 2): [0.0, 0.0, 0.0, 0.0]
q-values for state (1, 3): [-1.3737014638823901, -1.107223837634865, -7.9519393274569286, -0.23425187123790336]
q-values for state (1, 4): [-0.9525691641586566, 6.186348403994621, -1.5917377270037696, -0.5197863720163445]
q-values for state (2, 0): [-3.3133895490720913, -3.265129534794049, -3.2527829199211813, -11.191770165703375]
q-values for state (2, 1): [0.0, 0.0, 0.0, 0.0]
q-values for state (2, 2): [-7.5063566634172405, -1.2778691669524664, -9.14128257878115, -0.08109538903281402]
q-values for state (2, 3): [-1.4791146488748146, -6.586331779461695, -1.1840071588378192, 3.0598099017147904]
q-values for state (2, 4): [-0.529797799805102, 7.997592952261064, -0.8913002383216303, 1.8991417462547133]
q-values for state (3, 0): [-3.0789227353495736, -3.0700411687736704, -3.07167486459777, -11.014165756981942]
q-values for state (3, 1): [0.0, 0.0, 0.0, 0.0]
q-values for state (3, 2): [-1.5987201723501772, -1.8911533537320304, -9.99707245263115, -9.449035836861873]
q-values for state (3, 3): [0.0, 0.0, 0.0, 0.0]
q-values for state (3, 4): [1.9044565332746752, 9.999734386011127, -3.075626532104479, 4.174734555271477]
q-values for state (4, 0): [-2.8623843435159078, -2.8567682589501153, -2.8189165780418373, -2.8481337577563397]
q-values for state (4, 1): [-9.77295247236109, -2.548323989125602, -2.564950871408346, -2.5298003437763956]
q-values for state (4, 2): [-2.1982016198292733, -2.2302582507469966, -2.209919018362235, -10.075419280502166]
q-values for state (4, 3): [0.0, 0.0, 0.0, 0.0]
q-values for state (4, 4): [0.0, 0.0, 0.0, 0.0]
Optimal Policy Grid:
→→→→↓
←X X →↓
←X →→↓
↓X ↑X ↓
←→↑X G

```

Figure 4

EXERCISE 4:

For exercises 4 to 6 I had to complete a template given by the professor. I had to complete `execute_action` function by implementing the case where the robot executes charge action, in which case energy drive value is modified. Also, a manual policy to save energy when starting at (0,0) was implemented as can be seen in Figure 5.

```

# --- EXERCISE 4: MANUAL POLICY DEFINITION ---
# Goal: Define the optimal policy to minimize the energy deficit (Drive).
# The path must be: (0, 0) -> (0, 3), and at (0, 3) the action must be 'Charge'.

# 1. Path to the Charger: Down (↓) is index 1.
# Q-table index is (x, y, action)
q_table[0, 0, 1] = 1.0
q_table[0, 1, 1] = 2.0
q_table[0, 2, 1] = 3.0

# 2. Action at the Charger: Charge (🔌) is index 4.
q_table[0, 3, 4] = 10.0
# --- END OF MANUAL POLICY ---|

```

Figure 5

With this implementation, I got an optimal policy to reach the charger trying to save energy, this policy can be seen in Figure 6.

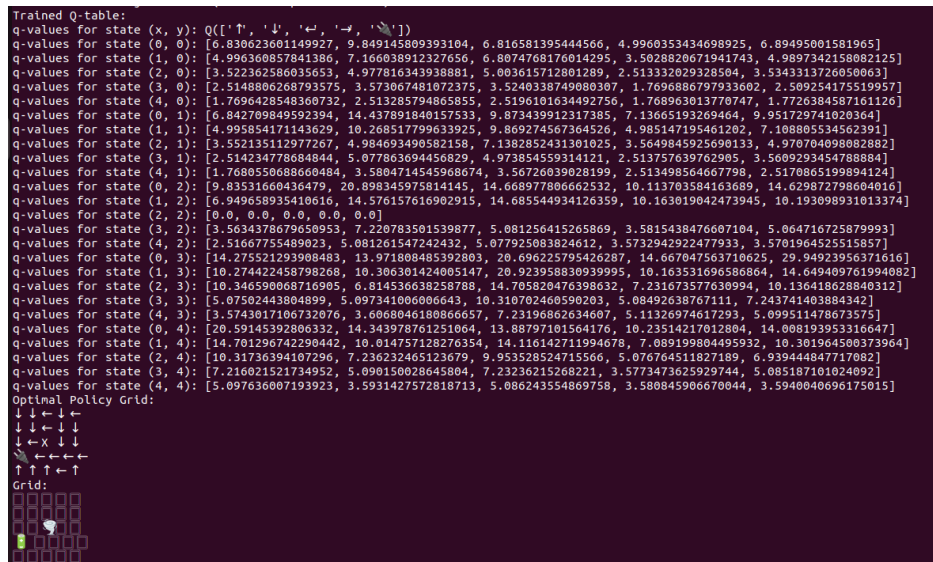


Figure 6

EXERCISE 5:

In this exercise, our robot needs to learn how to reach the charger but also saving energy. Differently from the previous exercise, in this case the robot will get a positive reward if energy drive value is decreased. This means that the robot needed to decide the best action without a established policy for the start position as in exercise 4.

In Figure 7 we can see the Q-table and optimal policy for this part of the practice. We can see some different values than in Figure 6 and some different decisions according to the optimal policy to reach the charger.

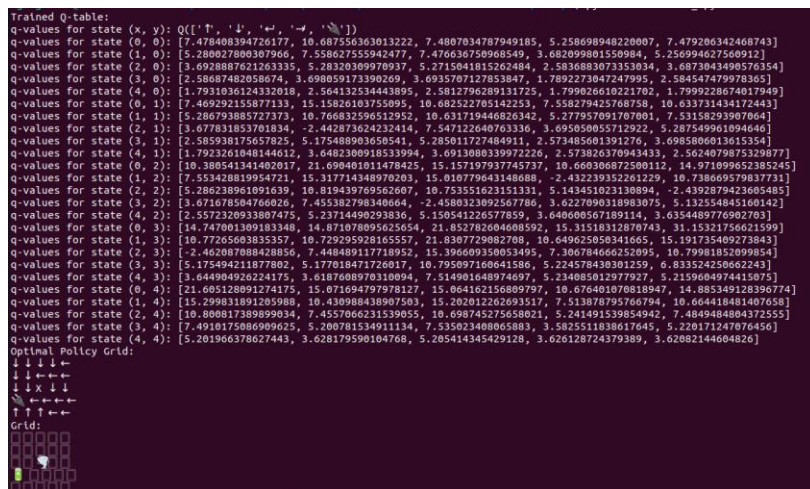


Figure 7

EXERCISE 6:

For this last exercise of the second main activity, I implemented two different motivations, one for charging and another for interacting with humans.

If we look at Figure 8, Q-table and optimal policy, we can see that the robot gives more importance to interacting with humans. This is because when the robot interacts the drive value of interacting is completely reduced, so the robot may see it as the most dominant motivation.

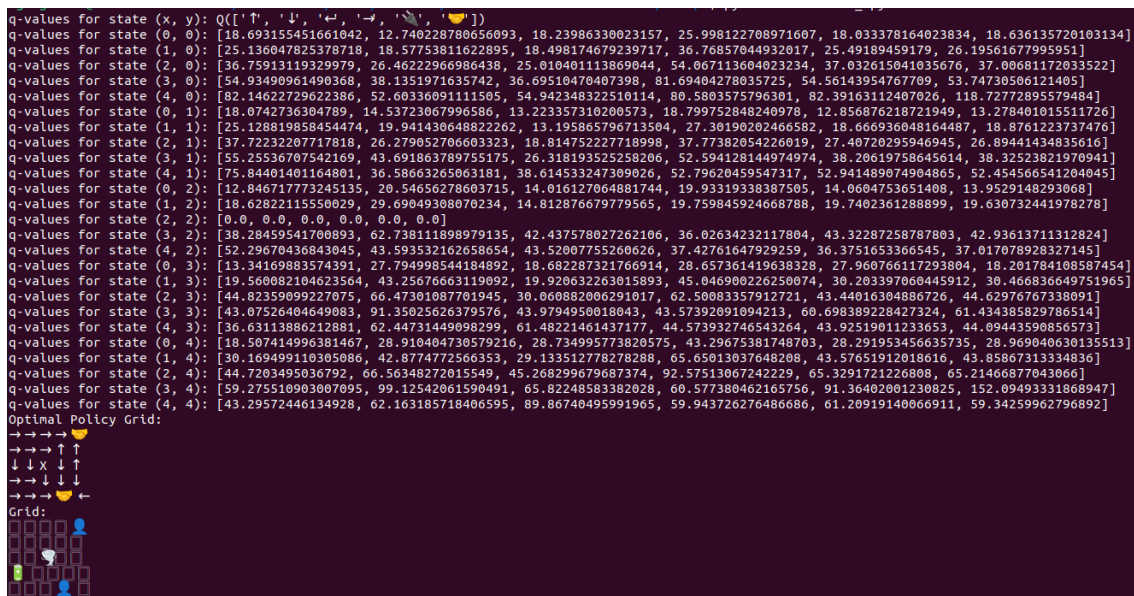


Figure 8

EXERCISE 7 (OPTIONAL):

For this last programming exercise, I needed help from Generative Tools as it is the most difficult. For achieving the objective, I implemented dynamic ϵ -greedy and also created two q-tables, one for recharge and another for socialize.

Depending on the dominant motivation which is calculated with *get_dominant_motivation* the robot will decide which actions to execute in order to reach the goal in each case. In Figure 9 we can see the optimal policy for each case. However, there may be a problem in the case where the motivation is SOCIALIZE as the robot prefers going to the

top right human. A possible solution for this conflict may be to increase the number of iterations so that the robot deeply learns the best path.

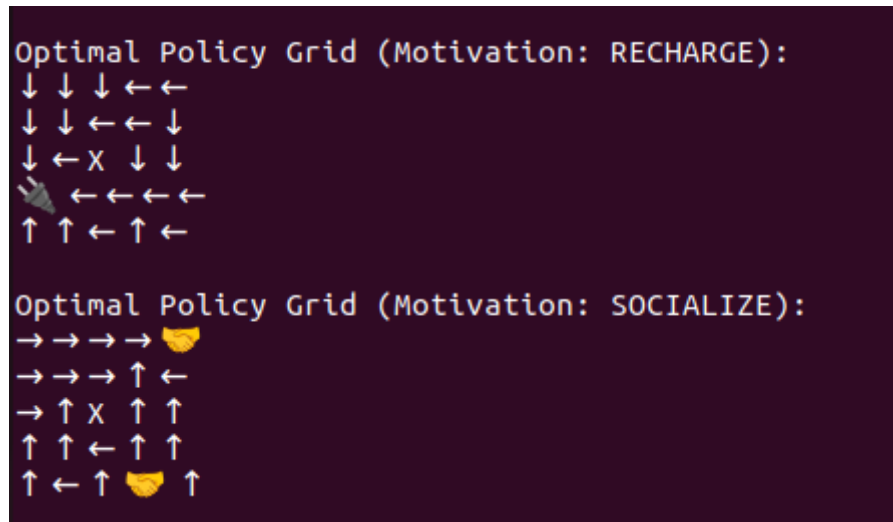


Figure 9

EXERCISE 8 (OPTIONAL):

The main drawbacks of applying Q-learning (QL) to real-world robots include issues related to action control, data inefficiency, and safety.

1. Continuous Action Spaces: Q-learning is fundamentally designed for discrete actions. However, real robots require continuous output such as velocity.
2. High Sample Complexity: Q-learning demands a lot of interactions to converge, making data collection slow and costly for physical hardware.
3. Exploration Risk: Some exploration strategies like ϵ -greedy force the robot to try random actions, which in some cases may be dangerous.
4. Deterministic Policies: QL typically learns a deterministic policy, which is often insufficient for tasks requiring stochasticity or flexibility to handle uncertainty and noise found in real-world sensor data and actuators.